

Towards a Fully Differentiable Framework for Autonomous Driving Based on Model-Structured Neural Networks

Sabrina Ciuffoletti, Gioele Defrancesco, Giovanni Scialla, Mattia Piazza,
Sebastiano Taddei, Gastone Pietro Rosati Papini

Abstract—Generalization in autonomous driving remains an open challenge, with current approaches divided between model-based pipelines, which ensure interpretability and safety, and end-to-end learning methods, which offer flexibility but limited guarantees. In this context, the proposed work establishes a hybrid framework that combines these paradigms through Model Structured Neural Networks (MSNNs), laying the groundwork for a fully differentiable architecture. The system integrates vehicle dynamics modeling, control, and motion planning into a unified pipeline. A structured neural forward model is first identified from data, followed by the learning of an inverse controller through a closed-loop, mental simulation approach. Moreover, a modular motion planning structure interfaces directly with the control layer, creating the structural groundwork for future joint optimization. The majority of the process is supported by *nnodely*, an open-source framework for the design and training of MSNNs. The paper presents the structure of each module and its integration within the control pipeline. The complete system is validated through closed-loop simulations on an FITENTH vehicle equipped with RTK-GPS, where the planning environment is directly connected to the inverse model for vehicle control. Results highlight the potential of structured neural approaches for achieving robust and generalizable autonomous driving.

I. INTRODUCTION

Achieving robust task generalization remains one of the central open challenges in autonomous driving and racing [1]. While recent advances in machine learning have enabled impressive perception and control capabilities, current systems still struggle when deployed in unseen environments or under distribution shifts [2]. This limitation has led to the emergence of two dominant and often contrasting paradigms.

On one side, modular architectures grounded in engineering principles and designed for interpretability and safety [1], [2]. On the other side, end-to-end learning approaches that leverage large-scale datasets to learn complex behaviors directly from experience [3]. Despite their respective successes, neither paradigm alone has yet demonstrated the ability to reliably generalize across the wide variability of real-world driving conditions [4].

In this context, hybrid approaches that combine model-based and learning methods are gaining increasing attention.

The authors are with the Department of Industrial Engineering, University of Trento, Via Sommarive 9, 38123 Povo (TN), Italy (email: gastone.rosatipapini@unitn.it)

This work was supported under the FIS 2 Call, Grant Assignment Decree No. 1236 adopted on 01/08/2023 by the Italian Ministry of University and Research (MUR), for the project FIS-2023-03684 'Structured neural network framework for modeling and control of autonomous systems – Neu4mes', CUP E53C24003800001.

Model Structured Neural Network (MSNN) offers a promising direction by embedding prior knowledge directly into neural architectures (*i.e.*, physical laws and control principles). This integration introduces inductive biases that improve data efficiency, interpretability, and robustness, while retaining the adaptability of learning-based methods [5], [6]. At the same time, a recent work on learning control through mental simulation has shown that it is possible to construct predictive and control models in a fully data-driven yet structured manner [7], by first learning forward dynamics and then synthesizing inverse models through differentiable simulation pipelines. These ideas suggest a pathway toward autonomous systems that are both learnable and controllable, bridging the gap between model-based and data-driven paradigms.

This paper builds on these insights to propose a unified, modular architecture for autonomous driving that bridges model-structured neural networks with classical control. The presented framework coordinates vehicle dynamics modeling, control, and motion planning within a unified development pipeline. First, a neural forward model of the vehicle dynamics is identified from data using a structured architecture that reflects the underlying physics. An inverse neural controller is then synthesized by training it in a closed loop with this forward model, following the mental simulation paradigm. To interface with this learning-based core, the high-level planner, inspired by modular motion planning approaches [7], [8], is introduced as a model-based module. The planner's underlying structure is designed for native compatibility with the *nnodely* framework¹, ensuring a standardized representation that allows the deterministic planning layer to be directly substituted or jointly trained with the control layer via the differentiable vehicle model with approaches inspired from [9] and [10].

A key enabler of this framework is *nnodely*, an open-source environment specifically designed for the development of MSNNs. By providing modular building blocks aligned with physical modeling and control concepts, *nnodely* allows these structured neural architectures to be trained, interconnected, and deployed within a unified workflow.

By embedding modularity directly within the learning process, this hybrid paradigm provides a clear path toward joint optimization across all levels of the autonomy stack. Ultimately, utilizing structured neural representations and differentiable simulation allows the proposed framework to

¹Available at <https://github.com/tonegas/nnodely>

move toward autonomous driving systems that are not only scalable and data-efficient but also robust, interpretable, and capable of generalizing beyond their training distribution.

II. UNIFIED PLANNING AND CONTROL FRAMEWORK

The overall architecture is composed of three main functional blocks: the vehicle model, the low-level control layer, and the motion planner, as illustrated in Fig. 1. To validate this architecture, we utilize a hybrid implementation: the lateral dynamics model and low-level lateral controller are fully realized as data-driven MSNNs within the nnodely framework (Section II-D), while the longitudinal control and high-level motion planner rely on deterministic, model-based methods. This modular interface provides a stable baseline for evaluation while ensuring that all layers share a standardized representation compatible with future joint training.

The pipeline is developed via a staged procedure. First, the vehicle model (Section II-A) is identified from data collected experimentally on the FITENTH vehicle while running predefined trajectories. Once the forward dynamics are learned, the low-level lateral controller (Section II-B) is synthesized by training it in a closed loop with the neural model following a differentiable training scheme.

Following training, the neural lateral controller is exported in ONNX format and deployed on the vehicle's Jetson platform alongside the C++ deterministic planner (Section II-C) for real-time closed-loop evaluation. This setup demonstrates the framework's capacity to seamlessly bridge learning-based control loops with classical, jerk-optimized planning structures.

A. Vehicle Model

The vehicle MSNN model for lateral dynamics relates the steering input angle to the resulting vehicle curvature. The network's physical structure is derived from the handling diagram (HD), assuming that the understeer factor K_{us} varies linearly with lateral acceleration ($a_y = \rho v_x^2$):

$$\rho(a_x, v_x, \delta) = \frac{\delta}{L + K_{us}(a_x)v_x^2}. \quad (1)$$

Here, ρ denotes the vehicle curvature, δ is the steering angle at the wheels, and L represents the wheelbase (inter-axle distance). The proposed neural network predicts the vehicle curvature at the next time step ρ_{k+1} (4) using a time window of $n = 30$ past steering samples, $\delta_k = [\delta_{k-n+1}, \dots, \delta_k]$, together with current longitudinal speed v_k and acceleration a_k . The model is structured as the product of a dynamic term (2) which captures the steering-to-curvature relationship through a speed-dependent *FIR filter* ($\mathbf{w}_\delta$ are the weights), and a parametric function with parameter A_i (3) that accounts for understeer effects under different longitudinal acceleration conditions. $\phi_{*i}(\cdot)$ denotes triangular fuzzy membership function. The number of channels is $N_v = 8$,

$N_a = 5$. The neural model's equations are:

$$D_v(v_k, \delta_k) = \sum_{i=1}^{N_v} \phi_{v_i}(v_k) (\mathbf{w}_{\delta_i}^\top \delta_k), \quad (2)$$

$$F_a(a_k, v_k) = \sum_{i=1}^{N_a} \phi_{a_i}(a_k) \frac{1}{1 + A_i v_k^2}, \quad (3)$$

$$\rho_{k+1} = D_v(v_k, \delta_k) F_a(a_k, v_k). \quad (4)$$

B. Lateral Control

The controller receives two future windows of length $n = 30$ for the target velocity $\mathbf{v}_{t_k}$ and curvature ρ_{t_k} , along with a past window of realized steering angles $\delta_{c_k} = [\delta_{k-n+1}, \dots, \delta_{k-1}]$ to account for initial steering conditions. A parametric function (6), using parameters A_i learned by the forward model, compensates for understeer effects, while a FIR filter (5) ($\mathbf{w}_\rho$) captures the vehicle's dynamic response. The predicted steering angle $\tilde{\delta}_k$ (7) is obtained as the sum of the target trajectory contribution and the past steering history.

$$K_t(a_k, \mathbf{v}_{t_k}, \rho_{t_k}) = \sum_{i=1}^{N_v} \phi_{v_i}(v_k) (\mathbf{w}_{\rho_i}^\top \nabla(a_k, \mathbf{v}_{t_k}, \rho_{t_k})) \quad (5)$$

$$\nabla(a_k, \mathbf{v}_{t_k}, \rho_{t_k}) = \rho_{t_k} \odot \left(1 + \sum_{i=1}^{N_a} \phi_{a_i}(a_k) A_i v_{t_k}^2 \right) \quad (6)$$

$$\tilde{\delta}_k = K_t(a_k, v_k, \mathbf{v}_{t_k}, \rho_{t_k}) + \mathbf{w}_{\delta_c}^\top \delta_{c_k} \quad (7)$$

C. Motion Planner

The motion planner is a jerk-optimized layer designed to generate smooth, high-fidelity trajectories. At its core, this layer solves a constrained optimization problem to minimize control effort, defined as $J = \int_0^{t_f} \mathbf{u}(t)^2 dt$, where the control input $\mathbf{u}(t)$ corresponds to longitudinal jerk or angular acceleration, similar to [11]. By formulating the motion primitives as the result of an Optimal Control Problem (OCP), the architecture can produce solutions that are inherently compliant with the system dynamics and boundary conditions.

While these primitives can be derived through analytical optimality conditions, the framework is designed to be independent of the underlying solver, as in [12], [8]. This allows the motion planning layer to be implemented either through closed-form analytical solutions or via structured neural networks trained to approximate the optimal control manifold.

Regardless of the implementation, the resulting primitives serve as a basis for the decision-making layer. By evaluating a *salience* metric against these candidates, the system measures the quality and robustness of the trajectories relative to the environmental context, ensuring the final control output is both optimal and context-aware.

More specifically, the total salience is computed as a weighted sum of longitudinal and lateral contributions, designed to balance efficiency, comfort, and safety.

The longitudinal component consists of: (i) the maneuver duration from an initial condition to a final one; (ii) the OCP cost; and (iii) a velocity constraint penalty.

Similarly, the lateral salience accounts for: (i) the time to

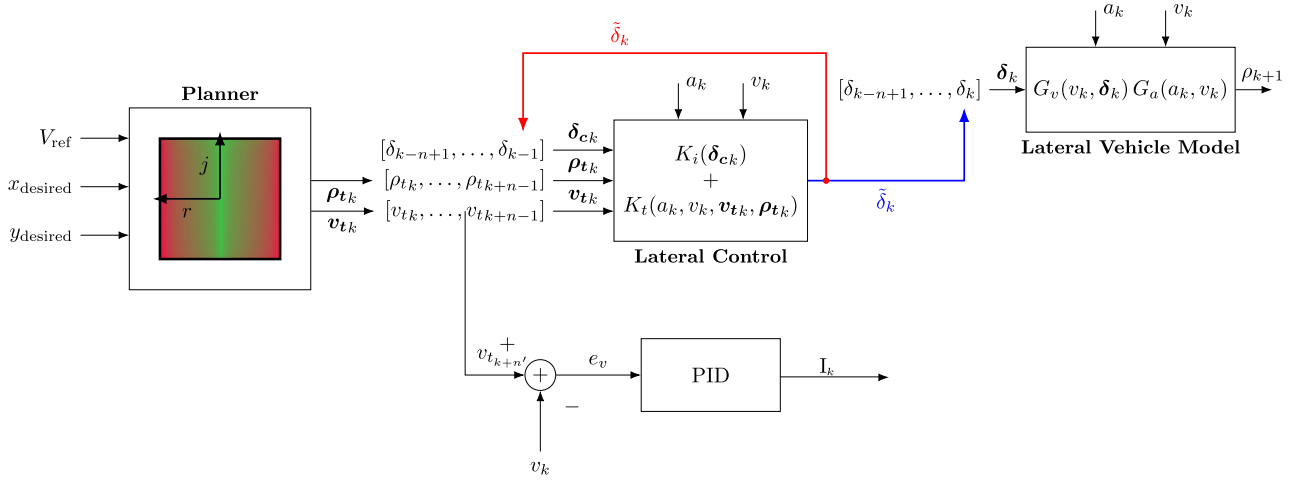


Fig. 1: Block diagram of the proposed pipeline: a minimum-jerk planner selects the optimal primitive from a jerk-based grid (green), given a lookahead target position ($x_{\text{desired}}, y_{\text{desired}}$) and reference speed computed according to the maximum road curvature (V_{ref}), and outputs future curvature ρ_{t_k} and velocity v_{t_k} profiles. A PID handles longitudinal control, while steering is generated by a neural network trained in a closed loop with a neural vehicle model. The blue line highlights the connection established between the Lateral Controller and the Model during the training phase. The red line highlights the Closed Loop that automatically performs the rolling window within the framework. Here, δ is used to represent the steer angle, whereas I_k is the current provided to the electric motor.

reach the road boundary; (ii) the time required to re-enter the bounds; and (iii) the initial lateral jerk r_0 .

To enable the management of complex environments, an additional term derived from a structured neural network can be integrated into this calculation. This learned component enables the planner to adapt to non-linear environmental features and sophisticated constraints that are difficult to model analytically, effectively refining the decision-making process in real-time.

Following the salience assignment, the *motor cortex* is constructed to provide a topographic organization of the motion space. In this structure, action pairs are arranged onto a two-dimensional grid, where each cell corresponds to a specific combination of longitudinal and lateral jerk, and its value represents the total salience of that maneuver. By placing similar actions in proximity, a structured map of the available motion space is created. This organization ensures the explainability of the planner, as the relationship between specific control inputs and their associated costs, such as those resulting from road boundaries, can be clearly visualized.

The final action selection is performed through a winner-take-all procedure, which identifies the grid cell with the minimum salience value. This process results in the selection of the most efficient and safe trajectory from the calculated options.

D. nnodely Framework

As mentioned earlier, the proposed architecture is implemented using nnodely, an open-source framework based on representing model-structured neural networks MSNNs as explicit structured representations of mathematical models. Rather than treating neural networks as opaque computation graphs, nnodely formalizes their structure, parameters, and

temporal dependencies through a unified abstraction that makes architectural choices explicit and interpretable.

A key feature of nnodely is the possibility to construct complex architectures through the composition of multiple sub-models. Individual components, such as the vehicle forward model and the inverse controller, can be defined and trained independently, and later interconnected through differentiable operations. In particular, the framework provides native support for *closed loop* connections, enabling the realization of recurrent and feedback structures required for control synthesis via mental simulation. This capability is central to the proposed approach, where the inverse controller is trained by embedding it within the learned forward dynamics and optimizing the closed-loop behavior.

From a modeling perspective, nnodely provides a set of domain-aligned building blocks that directly map to the components introduced in this paper. The vehicle model is naturally expressed through a combination of *FIR filters*, capturing temporal dynamics, and *parametric functions* encoding physically-inspired relations such as the handling diagram. Similarly, the controller structure uses the same primitives, ensuring architectural consistency between forward and inverse models. The use of *LocalModel* and *Fuzzify* operators enables the representation of nonlinear and regime-dependent behaviors through smooth interpolation across operating conditions. In the present work, recurrent training plays a crucial role, as it allows optimization over multi-step rollouts and enables the learning of control policies through differentiable simulation. Temporal coherence is preserved through dedicated data handling mechanisms, where input signals are organized into consistent time windows that match the structure of the model. This is essential for capturing the dynamic nature of the system and ensuring stability during closed-loop training.



Fig. 2: F1TENTH vehicle used for the experimental results.

Finally, nnody provides a complete pipeline from model definition to deployment, including dataset construction, training, validation, and export. Models can be serialized into a framework-independent representation and deployed in external environments. This end-to-end workflow ensures reproducibility and facilitates the integration of structured neural components into real-world control systems.

III. EXPERIMENTAL RESULTS

A. Current Experimental Setup

To evaluate the performance of the proposed framework, the hybrid planning and control architecture was validated on an autonomous F1TENTH vehicle. The experimental setup integrates the nnody framework with real-time onboard control execution, where the learning-based lateral modules, namely the forward vehicle model and the low-level inverse controller, operate in closed loop alongside the deterministic longitudinal controller and the jerk-optimized motion planner.

B. Case study

This case study evaluates the system under standard operating conditions. To accommodate the scaled physical dimensions of the F1TENTH platform, the experimental track was designed with smooth curvature transitions and moderate turning radii, enabling validation under low-speed driving conditions representative of standard path-following tasks. The target speed is set to $V_{target} = 1.7 \text{ m s}^{-1}$, as it is possible to see from 4a. The range of speeds and requested maneuvers is intentionally limited, as several simplifying assumptions are introduced for this preliminary analysis. Additionally, the lateral controller has been trained to handle higher acceleration demands, ensuring adequate responsiveness under these conditions. Overall, this case study demonstrates the full pipeline operating cohesively and provides a foundation for discussing future developments and potential extensions of the approach.

Figure 3 illustrates how the planner evaluates potential maneuvers by mapping salience across the two-dimensional action space of longitudinal jerk (j) and lateral jerk (r). As shown in the figure, a distinct, narrow vertical green corridor of low cost values highlights the optimal choices for the

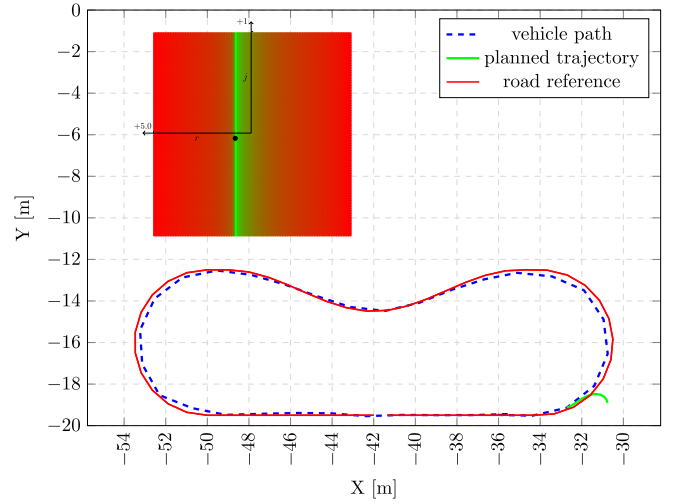


Fig. 3: Realized vehicle trajectory and visualization of action selection in the motor cortex.

vehicle. This sharp channel indicates that the system is highly sensitive to lateral maneuvers (r), heavily penalizing any deviations (shown in red) to keep the vehicle safely within the track boundaries. The final chosen action, represented by the black dot, sits within this optimal corridor to ensure a safe trajectory that aligns with the road geometry while decreasing the longitudinal speed as requested by the high-level safety constraints.

The real-world tracking capabilities resulting from these choices are confirmed by the 2D spatial trajectory layout, which depicts the scaled vehicle path successfully following the closed-loop track reference. To visualize the decision-making process, the planned forward-looking trajectory is also displayed. In the immediate vicinity of the vehicle, the planner attempts to return the vehicle to the reference path while simultaneously steering left to follow the upcoming curve. Although the final points of the planned trajectory extend beyond the road boundaries, likely due to the assumption of steady-state curvature, this introduces no operational issues thanks to the moving-horizon nature of the planner. For visualization purposes, the track coordinates are translated along both the x and y axes to display the data relative to a local origin of the GPS coordinates.

To see how these choices affect the vehicle's state over time, Fig. 4a illustrates the framework's ability to maintain specific velocity limits on physical hardware. The vehicle speed successfully tracks the varying target profile, hovering between 1.6 and 1.7 m s^{-1} , and intentionally drops to preserve lateral stability during tight cornering maneuvers, such as those at 8 s and 16 s . The forward-projected window of the chosen longitudinal motion primitive is explicitly visualized by the green trajectory, showing a progressively decreasing behaviour due to the requested vehicle speed. Even in this case, because these target states are continuously refreshed at each sampling interval, instantaneous tracking errors do not compromise the overall stability of the physical vehicle.

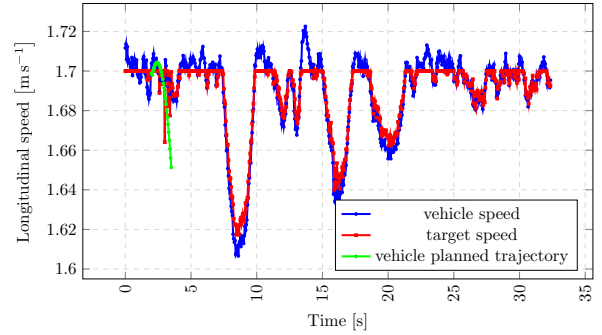
The lateral performance of the planner is further verified

by the vehicle curvature κ and steering angle δ profiles, shown in Figures 4b and 4c. Similar to the previous states, the forward-integrated curvature trajectory of the selected primitive is overlaid on the plot. The behavior is coherent with the 2D spatial trajectory, capturing the moment the vehicle reduces its curvature. Due to the simplification or truncation of higher-order terms in the primitive generation, this planned profile results a linear trajectory over the prediction horizon. As expected, the macroscopic behavior of these measured signals closely tracks the underlying road structure. However, both the physical steering and curvature signals exhibit high-frequency oscillations overlaid on the lower-frequency maneuver profiles. In real ground experiments on the F1TENTH platform, this behavior is primarily attributable to three factors. First, the physical constraints of small-scale hardware, such as mechanical play in the steering linkages and actuator deadband, introduce continuous micro-corrections around the target path. Second, unmodeled high-frequency dynamics, localized tire slip, and sensor noise inherent to physical deployments inherently challenge the low-level lateral controller. Finally, the discrete nature of the planning layer itself introduces a jittery characteristic; due to a lack of hysteresis in the action selection process, the winner-take-all mechanism can rapidly alternate between adjacent motion primitives with similar salience values, injecting high-frequency tracking demands into the downstream controller.

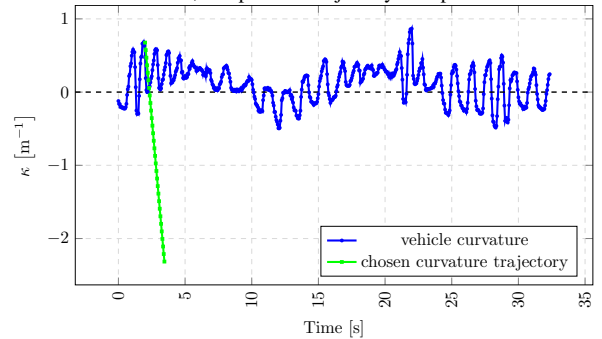
To alleviate these oscillations, future iterations will focus on implementing a more stable action selection process, alongside jointly training the controller and planner parameters in a fully closed loop. Introducing specific regularizing penalties, such as costs on the steering angle derivative ($\dot{\delta}$), into the objective function will explicitly enforce smoother physical control outputs. Furthermore, leveraging a structured neural network to adaptively update key planning parameters (e.g., speed-dependent lookahead distances and grid resolution) will optimize computational resources and smooth out the generated motion primitive profiles under variable operating conditions. This adaptive tuning is crucial in cross-scale deployment scenarios, where planning configurations optimized for full-scale vehicles must be dynamically re-scaled to match the severe geometric and inertial constraints of the F1TENTH platform.

IV. CONCLUSIONS AND FUTURE WORK

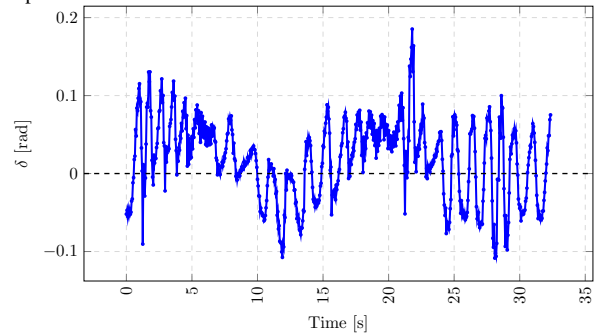
This work presented a first experimental validation of a hybrid framework for autonomous driving based on Model Structured Neural Networks, focusing on the integration of a model based motion planning and low-level differentiable control within a unified architecture. Evaluated on an F1TENTH autonomous vehicle, the architecture demonstrated encouraging real-world performance and highlighted several pathways for future optimization. A primary focus of the future work will be on extending the nnodly-based implementation to the entire control pipeline, including the motion planning layer. This will enable the realization of a fully unified and differentiable architecture, where all modules



(a). Comparison between target longitudinal velocity and achieved value. For time $T=2s$, the planned trajectory is reported



(b). Vehicle curvature. For time $T=2s$, the planned trajectory is reported



(c). Controlled steering angle

Fig. 4: Simulation results

are trained within a common framework. Such an approach would allow multi-level training strategies, supporting both independent module optimization and end-to-end fine-tuning across planning and control. Another key research direction concerns online adaptation. Since the proposed controllers are based on neural networks, they naturally enable the possibility of adapting their behavior during operation. Investigating robust and safe online learning mechanisms will be essential to further improve generalization capabilities and allow the system to cope with previously unseen conditions.

Overall, this work represents an initial step toward structured, learnable, and adaptable autonomous driving systems, highlighting the potential of MSNN-based approaches for bridging modular and end-to-end paradigms.

REFERENCES

- [1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.
- [2] J. Zhao, W. Zhao, B. Deng, Z. Wang, F. Zhang, W. Zheng, W. Cao, J. Nan, Y. Lian, and A. F. Burke, "Autonomous driving system: A comprehensive survey," *Expert Systems with Applications*, vol. 242, p. 122836, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423033389>
- [3] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10 164–10 183, 2024.
- [4] J. Hu, Y. Wang, S. Cheng, J. Xu, N. Wang, B. Fu, Z. Ning, J. Li, H. Chen, C. Feng *et al.*, "A survey of decision-making and planning methods for self-driving vehicles," *Frontiers in Neurobotics*, vol. 19, p. 1451923, 2025.
- [5] M. Da Lio, D. Bortoluzzi, and G. P. Rosati Papini, "Modelling longitudinal vehicle dynamics with neural networks," *Vehicle System Dynamics*, vol. 58, no. 11, pp. 1675–1693, 2020. [Online]. Available: <https://doi.org/10.1080/00423114.2019.1638947>
- [6] M. Piccinini, S. Taddei, M. Larcher, M. Piazza, and F. Biral, "A physics-driven artificial agent for online time-optimal vehicle motion planning and control," *IEEE Access*, vol. 11, pp. 46 344–46 372, 2023.
- [7] M. Da Lio, R. Donà, G. P. Rosati Papini, F. Biral, and H. Svensson, "A mental simulation approach for learning neural-network predictive control (in self-driving cars)," *IEEE Access*, vol. 8, pp. 192 041–192 064, 2020.
- [8] M. Piazza, M. Piccinini, S. Taddei, and F. Biral, "MPTree: A sampling-based vehicle motion planner for real-time obstacle avoidance," *IFAC-PapersOnLine*, vol. 58, no. 10, pp. 146–153, 2024.
- [9] F. Jahncke, B. Zarrouki, M. Piccinini, J. D'sa, D. Isele, S. Bae, and J. Betz, "Differentiable weights-varying nonlinear mpc via gradient-based policy learning: An autonomous vehicle guidance example," *IEEE Robotics and Automation Letters*, 2026.
- [10] P. Karkus, B. Ivanovic, S. Mannor, and M. Pavone, "Diffstack: A differentiable and modular control stack for autonomous vehicles," in *Conference on robot learning*. PMLR, 2023, pp. 2170–2180.
- [11] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a frenet frame," in *2010 IEEE international conference on robotics and automation*. IEEE, 2010, pp. 987–993.
- [12] R. Trauth, K. Moller, G. Würsching, and J. Betz, "Frenetix: A high-performance and modular motion planning framework for autonomous driving," *IEEE Access*, 2024.